DeepFP: Deep-Unfolded Fractional Programming for MIMO Beamforming

Jianhang Zhu, *Graduate Student Member, IEEE*, Tsung-Hui Chang, *Fellow, IEEE*, Liyao Xiang, *Member, IEEE*, and Kaiming Shen, *Senior Member, IEEE*

arXiv:2601.02822v1 [cs.IT] 6 Jan 2026

***Abstract*—This work proposes a mixed learning-based and optimization-based approach to the weighted-sum-rates beamforming problem in a multiple-input multiple-output (MIMO) wireless network. The conventional methods, i.e., the fractional programming (FP) method and the weighted minimum mean square error (WMMSE) algorithm, can be computationally demanding for two reasons: (i) they require inverting a sequence of matrices whose sizes are proportional to the number of antennas; (ii) they require tuning a set of Lagrange multipliers to account for the power constraints. The recently proposed method called the reduced WMMSE addresses the above two issues for a single cell. In contrast, for the multicell case, another recent method called the FastFP eliminates the large matrix inversion and the Lagrange multipliers by using an improved FP technique, but the update stepsize in the FastFP can be difficult to decide. As such, we propose integrating the deep unfolding network into the FastFP for the stepsize optimization. Numerical experiments show that the proposed method is much more efficient than the learning method based on the WMMSE algorithm.**

***Index Terms*—Multiple-input multiple-output (MIMO) beamforming, weighted sum rates maximization, deep unfolding.**

I. Introduction

A fundamental problem of multiple-input-multiple-output (MIMO) system design is to optimize the transmit beamformers to maximize the weighted-sum-rates (WSR) throughout cellular networks, namely the WSR problem. The weighted minimum mean square error (WMMSE) algorithm [2], [3] and the fractional programming (FP) [4], [5] constitute two popular approaches in this area. Since two methods are both iteratively structured, a natural idea is to learn their behaviors via deep unfolding, as pursued extensively in [6]–[13]. However, two main challenges arise when it comes to the multicell MIMO case: (i) the iterative algorithm (e.g., WMMSE or FP) requires inverting large matrices, yet the matrix inversion is much more difficult to learn than the matrix addition and multiplication; (ii) the iterative algorithm requires finding the optimal Lagrange multipliers for the power constraint, which is complicated and highly nonlinear and can increase the training cost considerably. To address these issues, this paper proposes a novel deep unfolding scheme called *DeepFP* that relies on the new FP technology. Differing from the previous methods [6]–[13], DeepFP avoids learning the large matrix inversion and the nonlinear optimization of Lagrange multipliers, and only focuses on how to coordinate a small set of scalar stepsizes. Here is a big picture of how this work is developed. The original objective function for the beamforming problem is f_o, which is difficult to tackle directly. The conventional FP method [4], [5] suggests converting f_o to f_q so that the iterative optimization is easy to perform, but then the new issue is that it entails computing the large matrix inverse. To get rid of this complexity, a more recent method called the nonhomogeneous quadratic transform [14] further converts f_q to f_n, but then its performance is sensitive to the choice of stepsize: if the stepsize is too large, then the iteration may not converge; if the stepsize is too small, then the convergence would slow down. To decide the stepsize, [14] imposes a strong assumption that all the users in the same cell use the same stepsize, and then shows that the stepsize is upper bounded by some eigenvalue. In contrast, this paper considers tuning the stepsize separately for each individual user across the network, so this eigenvalue-based upper bound disappears, and thus the choice of stepsize can be more aggressive.

The WSR problem is notoriously difficult. In fact, it is shown to be NP-hard even for the single-input-single-output case [15]. Aside from the branch-and-bound approaches in [16], [17], most existing works aim to find a local optimum efficiently. The classic methods include the maximum ratio transmission (MRT) [18], the zero-forcing (ZF) method [19], and the regularized ZF precoding (RZF) method [20], which are verified at the link level under certain conditions but can lead to quite large performance losses at the system level. In more recent literature, the WMMSE algorithm [2], [3] is widely adopted for solving the WSR problem. Its main idea is to utilize a connection between the rate maximization and the mean square error (MSE) minimization to rewrite the WSR problem as a weighted MSE minimization problem—which can be efficiently solved by the block coordinate descent (BCD) method [21] in an iterative fashion. Thanks to the BCD theory, the WMMSE algorithm has provable convergence to a stationary point solution of the WSR problem.

However, the WMMSE algorithm can incur high computational tension in the multicell MIMO case. To be more specific,

Manuscript accepted to IEEE Transactions on Communications on January 3, 2026. The work of Jianhang Zhu and Kaiming Shen was supported in part by the NSFC under Grant 12426306 and in part by Guangdong Basic and Applied Basic Research under Grant 2023B0303000001. The work of Liyao Xiang was supported by the NSFC under Grant U25A20445 and Grant 62272306. An earlier version of this paper has been presented in part at IEEE SPAWC 2025 [1]. Source codes are available at https://github.com/zhujhz/DeepFP.git. *(Corresponding author: Kaiming Shen.)*

Jianhang Zhu and Kaiming Shen are with the School of Science and Engineering, The Chinese University of Hong Kong (Shenzhen), 518172 Shenzhen, China (e-mail: jianhangzhu1@link.cuhk.edu.cn; shenkaiming@cuhk.edu.cn).

Tsung-Hui Chang is with the School of Artificial Intelligence, The Chinese University of Hong Kong (Shenzhen), 518172 Shenzhen, China (e-mail: tsunghui.chang@ieee.org).

Liyao Xiang is with Shanghai Jiao Tong University, 200240, Shanghai, China (e-mail: xiangliyao08@sjtu.edu.cn).

TABLE I
COMPARISON OF THE DIFFERENT METHODS FOR MIMO BEAMFORMING WHEN EACH TRANSMITTER HAS N ANTENNAS.

Method	Can avoid $N \times N$ matrix inversion?	Can avoid Lagrange multipliers tuning?	Can work for multiple cells?
WMMSE [2], [3]	✗	✗	✓
Reduced WMMSE [22]	✗ (but can reduce the matrix size)	✓	✗
FP [4]	✗	✗	✓
Deep unfolding + FP [6]	✗	✓	✓
FastFP [14], [23]	✓ (but requires eigencomputation)	✓	✓
Proposed DeepFP	✓	✓	✓

each iterate of WMMSE requires inverting a matrix whose size is proportional to the number of transmit antennas. Thus, in the multicell MIMO case with a large number of antennas deployed at the transmitter side, the WMMSE algorithm requires lots of large matrix inversions. Such tension has been relieved more or less by a recent work [22]. The main idea of [22] is to recast the beamforming vectors to a new space whose dimension only depends on the total number of receive antennas (or the number of users, assuming each user has only one receive antenna). This modified WMMSE algorithm (referred to as the RWMMSE in [22]) now instead inverts matrices whose sizes are proportional to the number of users. Clearly, the RWMMSE algorithm has reduced complexity only when there are a limited number of users in the network.

Another challenge faced by the WMMSE algorithm in multicell MIMO is caused by the power constraint. Specifically, in each iteration, WMMSE needs to determine a Lagrange multiplier for each cell to satisfy the power constraint on the beamforming vectors. The optimal Lagrange multiplier has no closed-form solution and is typically addressed via bisection search [2]. The recently proposed "reduced WMMSE" algorithm [22] has partially addressed this issue. The authors of [22] show that it is optimal to scale all the beamforming vectors simultaneously to meet the power constraint when considering a single cell. However, it is difficult to extend the above result for multiple cells. Another approach to the multicell MIMO beamforming problem is based on the manifold optimization [24]. Its main idea is to restrict the beamforming variables to a Riemannian manifold defined by the power constraint, thereby converting the constrained optimization to the unconstrained. However, the manifold method only optimizes beamforming vectors under the fixed power levels, whereas WMMSE can optimize beamforming vectors and powers jointly. Moreover, its performance is verified only for the single-cell network.

Aside from the above model-driven method, there is a surge of research interest in the data-driven approach to the multicell MIMO beamforming problem. Differing from those pure black-box learning methods [25]–[28] that attempt to mimic the existing optimization methods (e.g., WMMSE) via the universal approximation of capability deep neural network (DNN), the deep unfolding methods [29]–[31] take into account the iterative structure of the conventional model-driven algorithms and aim to learn the behavior of each iteratation. For the WSR beamforming problem, the previous studies [6]–[13] mostly take the WMMSE algorithm as the learning target of deep unfolding. For example, the deep unfolding network in [6] aims at the RWMMSE algorithm, while [7], [8] aim at the WMMSE algorithm. However, as these deep unfolding methods successfully mimic WMMSE, they in the meanwhile inherit the aforementioned drawbacks of their target algorithms. As such, the deep unfolding network in [6] can only handle a single cell, [8] has to approximate the large matrix inversion in a suboptimal approximate fashion, and [7] is limited to the multiple-input-single-output (MISO) case in order to avoid learning the bisection search for the optimal Lagrange multipliers.

To overcome the above bottleneck, the deep unfolding method proposed in this paper takes advantage of an intimate connection between WMMSE and FP. Roughly speaking, FP refers to a class of optimization problems which are fractionally structured, e.g., the sum-of-ratios maximization. It turns out that the WSR problem can be recast to a sum-of-ratios problem, and accordingly the WMMSE algorithm boils down to a special case of the FP algorithm [4], [5]. In fact, the large matrix inversion and the Lagrange multiplier optimization have been well studied in the realm of FP, e.g., the so-called *nonhomogeneous quadratic transform* [14], [23] can address both issues. Thus, unlike the previous works [6]–[13] that consider deep-unfolding the WMMSE algorithm directly, this work proposes incorporating the inhomogeneous quadratic transform into the deep unfolding paradigm. We then show that the core of the learning task is to decide the stepsize used in the inhomogeneous quadratic transform-based FP. The main features and advantages of the proposed method are summarized in Table I. Our work introduces two main novelties:

- *A New Paradigm of Deep-Unfolding:* The existing deep-unfolding methods for beamforming typically mimic the behavior of the traditional FP (based on the quadratic transform) or the WMMSE algorithms. In contrast, this work builds upon the nonhomogeneous quadratic transform, which inherently avoids the complexity of computing the large matrix inverse. Furthermore, unlike the existing nonhomogeneous quadratic transform in [14] that tunes the stepsize parameter on a per-cell basis, this work novelly suggests optimizing the stepsize parameter for each individual user. The numerical results show that this new paradigm leads to faster convergence and superior performance.
- *A New Hybrid Training Strategy:* The proposed DeepFP employs a novel hybrid training strategy that combines supervised and unsupervised learning. Specifically, in the first stage, the DNN is initialized and trained using

$$f_q(\underline{\mathbf{V}},\underline{\mathbf{\Gamma}},\underline{\mathbf{Y}}) = \sum_{\ell,k}\Big[\operatorname{tr}\left(2\Re\left\{\mathbf{V}_{\ell k}^H\mathbf{\Lambda}_{\ell k}\right\} - \omega_{\ell k}\mathbf{Y}_{\ell k}^H\mathbf{D}_{\ell k}\mathbf{Y}_{\ell k}\left(\mathbf{I}_d+\mathbf{\Gamma}_{\ell k}\right)\right) + \omega_{\ell k}\log\left|\mathbf{I}_d+\mathbf{\Gamma}_{\ell k}\right| - \operatorname{tr}\left(\omega_{\ell k}\mathbf{\Gamma}_{\ell k}\right)\Big] \tag{7}$$

model-driven FastFP solutions as labels to ensure rapid convergence and robust initialization. Subsequently, in the second stage, the DNN is fine-tuned using the actual weighted sum-rate objective as the loss function, allowing the network to refine its performance toward the true objective.

The remainder of this paper is organized as follows. Section II introduces the weight sum-rate problem formulation. Section III shows and compares existing model-driven algorithms, including the FP algorithm and the FastFP algorithm. Section IV develops our proposed DeepFP network based on the FastFP algorithm. Section V presents numerical results. Finally, Section VI concludes the paper.

Here and throughout, bold lower-case letters represent vectors while bold upper-case letters represent matrices. For a vector $\mathbf{a}$, $\mathbf{a}^H$ is its conjugate transpose. For a matrix $\mathbf{A}$, $\mathbf{A}^H$ is its conjugate transpose and $\|\mathbf{A}\|_F$ is its Frobenius norm. $\operatorname{col}(\mathbf{A})$ refers to the number of columns in matrix A. For a square matrix $\mathbf{A}$, $\operatorname{tr}(\mathbf{A})$ is its trace, $|\mathbf{A}|$ is its determinant, and $\lambda_{\max}(\mathbf{A})$ is its largest eigenvalue. Denote by $\mathbf{I}_d$ the $d\times d$ identity matrix, $\mathbb{C}^n$ the set of $n\times 1$ vectors, $\mathbb{C}^{d\times n}$ the set of $d\times n$ matrices, and $\mathbb{H}_+^{d\times d}$ the set of $d\times d$ positive definite matrices. For a complex number $\mathbf{a}\in\mathbb{C}$, $\Re\{\mathbf{a}\}$ is its real part, $|\mathbf{a}|$ is its absolute value. The underlined letters represent the collections of the associated vectors or matrices, e.g., for $\mathbf{a}_1,\ldots,\mathbf{a}_n\in\mathbb{C}^d$ we write $\underline{\mathbf{a}} = [\mathbf{a}_1,\mathbf{a}_2,\ldots,\mathbf{a}_n]^\top\in\mathbb{C}^{n\times d}$.

II. Weighted Sum-Rate Maximization Problem

Consider a downlink multi-user multiple-input-multiple-output (MU-MIMO) system with L cells. Within each cell, one base station (BS) with N_t transmit antennas serves K users. The kth user in the ℓth cell is indexed as (ℓ,k). Assume that user (ℓ,k) has N_r receive antennas and that d data streams are intended for it. Let $\mathbf{V}_{\ell k}\in\mathbb{C}^{N_t\times d}$ represent the beamforming matrix used by BS ℓ associated with the signal $\mathbf{s}_{\ell k}\in\mathbb{C}^{d\times 1}$ for user (ℓ,k). Assuming that $\mathbb{E}[\mathbf{s}_{\ell k}\mathbf{s}_{\ell k}^H]=\mathbf{I}_d$, the received signal $y_{\ell k}$ at user (ℓ,k) is given by

$$\mathbf{y}_{\ell k} = \underbrace{\mathbf{H}_{\ell k,\ell}\mathbf{V}_{\ell k}\mathbf{s}_{\ell k}}_{\text{desired signal}} + \underbrace{\sum_{j=1,j\neq k}^{K}\mathbf{H}_{\ell k,\ell}\mathbf{V}_{\ell j}\mathbf{s}_{\ell j}}_{\text{intracell interference}} + \underbrace{\sum_{i=1,i\neq\ell}^{L}\sum_{j=1}^{K}\mathbf{H}_{\ell k,i}\mathbf{V}_{ij}\mathbf{s}_{ij}}_{\text{intercell interference}} + \mathbf{n}_{\ell k}, \tag{1}$$

where the channel state information (CSI) $\mathbf{H}_{\ell k,i}\in\mathbb{C}^{N_r\times N_t}$ is the channel from BS i to user (ℓ,k), and $\mathbf{n}_{\ell k}\sim\mathcal{CN}(\mathbf{0},\sigma^2\mathbf{I})$ is the additive white Gaussian noise with power level σ^2. The achievable data rate for user (ℓ,k) can be computed as [32]

$$R_{\ell k} = \log\left|\mathbf{I}+\mathbf{V}_{\ell k}^H\mathbf{H}_{\ell k,\ell}^H\mathbf{F}_{\ell k}^{-1}\mathbf{H}_{\ell k,\ell}\mathbf{V}_{\ell k}\right|, \tag{2}$$

Algorithm 1 FP for Multicell MIMO Beamforming

1: **input:** The current CSI.
2: Initialize $\underline{\mathbf{V}}$ to feasible values under the power constraint.
3: **repeat**
4: Update each $\mathbf{Y}_{\ell k}$ by (9).
5: Update each $\mathbf{\Gamma}_{\ell k}$ by (10).
6: Update each $\mathbf{V}_{\ell k}$ by (11).
7: **until** the objective value converges
8: **output:** Final beamforming matrix $\underline{\mathbf{V}}$

where

$$\mathbf{F}_{\ell k} = \sum_{j=1,j\neq k}^{K}\mathbf{H}_{\ell k,\ell}\mathbf{V}_{\ell j}\mathbf{V}_{\ell j}^H\mathbf{H}_{\ell k,\ell}^H + \sum_{i=1,i\neq\ell}^{L}\sum_{j=1}^{K}\mathbf{H}_{\ell k,i}\mathbf{V}_{ij}\mathbf{V}_{ij}^H\mathbf{H}_{\ell k,i}^H + \sigma^2\mathbf{I}_{N_r}. \tag{3}$$

We seek the optimal transmit beamformers $\underline{\mathbf{V}}$ to maximize the weighted sum rates:

$$\max_{\underline{\mathbf{V}}} \quad f_o(\underline{\mathbf{V}}) := \sum_{\ell=1}^{L}\sum_{k=1}^{K}w_{\ell k}R_{\ell k} \tag{4a}$$

$$\text{s.t.} \quad \sum_{k=1}^{K}\operatorname{tr}(\mathbf{V}_{\ell k}\mathbf{V}_{\ell k}^H)\le P_\ell,\ \ell=1,2,\ldots,L, \tag{4b}$$

where the nonnegative weight $w_{\ell k}\ge 0$ reflects the priority of user (ℓ,k), and the constant P_ℓ is the power budget of BS ℓ.

III. Existing Optimization-Based Methods

A. FP Method

By the Lagrangian dual transform [5], the original objective $f_o(\underline{\mathbf{V}})$ is converted to

$$f_r(\underline{\mathbf{V}},\underline{\mathbf{\Gamma}}) = \sum_{\ell=1}^{L}\sum_{k=1}^{K}w_{\ell k}\left[\log\left|\mathbf{I}_d+\mathbf{\Gamma}_{\ell k}\right| - \operatorname{tr}(\mathbf{\Gamma}_{\ell k}) + \operatorname{tr}((\mathbf{I}+\mathbf{\Gamma}_{\ell k})\mathbf{V}_{\ell k}^H\mathbf{H}_{\ell k,\ell}^H\mathbf{D}_{\ell k}^{-1}\mathbf{H}_{\ell k,\ell}\mathbf{V}_{\ell k})\right], \tag{5}$$

where

$$\mathbf{D}_{\ell k} = \sum_{i=1}^{L}\sum_{j=1}^{K}\mathbf{H}_{\ell k,i}\mathbf{V}_{ij}\mathbf{V}_{ij}^H\mathbf{H}_{\ell k,i}^H + \sigma^2\mathbf{I}_{N_r}. \tag{6}$$

The FP method then applies the quadratic transform [4] to further recast $f_o(\underline{\mathbf{V}})$ into $f_q(\underline{\mathbf{V}},\underline{\mathbf{\Gamma}},\underline{\mathbf{Y}})$ displayed in (7) with

$$\mathbf{\Lambda}_{\ell k} = w_{\ell k}\mathbf{H}_{\ell k,\ell}^H\mathbf{Y}_{\ell k}(\mathbf{I}_d+\mathbf{\Gamma}_{\ell k}). \tag{8}$$

The new objective $f_q(\underline{\mathbf{V}},\underline{\mathbf{\Gamma}},\underline{\mathbf{Y}})$ is separately concave in $\underline{\mathbf{V}},\underline{\mathbf{\Gamma}},\underline{\mathbf{Y}}$, so the FP algorithm allows iteratively optimizing these variables as

$$\mathbf{Y}_{\ell k} = \mathbf{D}_{\ell k}^{-1}\mathbf{H}_{\ell k,\ell}\mathbf{V}_{\ell k}, \tag{9}$$

$$\mathbf{\Gamma}_{\ell k} = \mathbf{V}_{\ell k}^H\mathbf{H}_{\ell k,\ell}^H\mathbf{F}_{\ell k}^{-1}\mathbf{H}_{\ell k,\ell}\mathbf{V}_{\ell k}, \tag{10}$$

$$\mathbf{V}_{\ell k} = (\eta_\ell\mathbf{I}_{N_t}+\mathbf{L}_\ell)^{-1}\mathbf{\Lambda}_{\ell k}, \tag{11}$$

$$f_n(\underline{\mathbf{V}},\underline{\mathbf{\Gamma}},\underline{\mathbf{Y}},\underline{\mathbf{Z}}) = \sum_{\ell,k}\Big[\,\mathrm{tr}\left(2\Re\left\{\mathbf{V}_{\ell k}^H\mathbf{\Lambda}_{\ell k}+\mathbf{V}_{\ell k}^H(\lambda_\ell\mathbf{I}_{N_t}-\mathbf{L}_\ell)\mathbf{Z}_{\ell k}\right\}+\mathbf{Z}_{\ell k}^H(\mathbf{L}_\ell-\lambda_\ell\mathbf{I}_{N_t})\mathbf{Z}_{\ell k}-\lambda_\ell\mathbf{V}_{\ell k}^H\mathbf{V}_{\ell k}\right)$$
$$-\mathrm{tr}\left(\omega_{\ell k}\sigma^2(\mathbf{I}_d+\mathbf{\Gamma}_{\ell k})\mathbf{Y}_{\ell k}^H\mathbf{Y}_{\ell k}\right)+\omega_{\ell k}\log|\mathbf{I}_d+\mathbf{\Gamma}_{\ell k}|-\mathrm{tr}\left(\omega_{\ell k}\mathbf{\Gamma}_{\ell k}\right)\Big] \quad (15)$$

where

$$\mathbf{L}_\ell=\sum_{i=1}^{L}\sum_{j=1}^{K}w_{ij}\mathbf{H}_{ij,\ell}^H\mathbf{Y}_{ij}(\mathbf{I}_d+\mathbf{\Gamma}_{ij})\mathbf{Y}_{ij}^H\mathbf{H}_{ij,\ell}, \quad (12)$$

and the Lagrange multiplier η_ℓ in (11) for the power constraint is computed as

$$\eta_\ell=\min\left\{\eta\ge 0:\sum_{k=1}^{K}\mathrm{tr}(\mathbf{V}_{\ell k}\mathbf{V}_{\ell k}^H)\le P_\ell\right\}, \quad (13)$$

as summarized in Algorithm 1. Note that the WMMSE algorithm [2], [3] is a special case of Algorithm 1.

B. FastFP Method [14], [23]

The main drawback of the FP method is that it requires computing the large matrix inverse in (11): recall that $\mathbf{L}_\ell$ is an $N_t\times N_t$ matrix and N_t is a large number in the multicell MIMO setting. To eliminate the large matrix inversion, we can incorporate the following bound into the FP method:

Lemma 1. *(Nonhomogeneous Bound [33]) Suppose that two Hermitian matrices $\mathbf{L},\mathbf{K}\in\mathbb{H}^{m\times m}$ satisfy $\mathbf{L}\preceq\mathbf{K}$. Then for any two matrices $\mathbf{X},\mathbf{Z}\in\mathbb{C}^{m\times m}$, one has*

$$\mathrm{tr}(\mathbf{X}^H\mathbf{L}\mathbf{X})\le\mathrm{tr}\left(\mathbf{X}^H\mathbf{K}\mathbf{X}+2\Re\{\mathbf{X}^H(\mathbf{L}-\mathbf{K})\mathbf{Z}\}+\mathbf{Z}^H(\mathbf{K}-\mathbf{L})\mathbf{Z}\right), \quad (14)$$

where the equality holds if $\mathbf{Z}=\mathbf{X}$.

Remark 1. *In a nutshell, we seek some matrix $\mathbf{L}$ in (14) that is easy to invert, so it is natural to let $\mathbf{L}=\lambda\mathbf{I}$. It remains to choose $\lambda\in\mathbb{R}$ to meet the condition $\mathbf{L}\preceq\mathbf{K}$. We can compute the largest eigenvalue of $\mathbf{K}$ and let $\lambda=\lambda_{\max}(\mathbf{L})$. An alternative is to let $\lambda=\|\mathbf{L}\|_F$, but the gap between $\mathbf{L}$ and $\mathbf{K}$ becomes larger, so the convergence slows down.*

In light of Lemma 1 and Remark 1, we further recast $f_q(\underline{\mathbf{V}},\underline{\mathbf{\Gamma}},\underline{\mathbf{Y}})$ into $f_n(\underline{\mathbf{V}},\underline{\mathbf{\Gamma}},\underline{\mathbf{Y}},\underline{\mathbf{Z}})$ as displayed in (15), where

$$\lambda_\ell=\lambda_{\max}(\mathbf{L}_\ell) \quad (16)$$

When other variables are held fixed, each $\underline{\mathbf{Z}}$ in (15) is optimally determined as

$$\mathbf{Z}_{\ell k}=\mathbf{V}_{\ell k}. \quad (17)$$

When other variables are fixed, each $\mathbf{V}_{\ell k}$ for the current iteration t is optimally determined based on that of the previous iteration $t-1$ as

$$\mathbf{V}_{\ell k}^{(\tau)}=\begin{cases}\widehat{\mathbf{V}}_{\ell k} & \text{if } \sum_{j=1}^{K}\|\widehat{\mathbf{V}}_{\ell j}\|_F^2\le P_\ell\\ \sqrt{\frac{P_\ell}{\sum_{j=1}^{K}\|\widehat{\mathbf{V}}_{\ell j}\|_F^2}}\widehat{\mathbf{V}}_{\ell k} & \text{otherwise,}\end{cases} \quad (18)$$

Algorithm 2 FastFP for Multicell MIMO Beamforming

1: **input:** The current CSI.
2: Initialize $\underline{\mathbf{V}}$ to feasible values under the power constraint.
3: **repeat**
4: Update each $\mathbf{Z}_{\ell k}$ by (17).
5: Update each $\mathbf{Y}_{\ell k}$ by (9).
6: Update each $\mathbf{\Gamma}_{\ell k}$ by (10).
7: Update each $\mathbf{V}_{\ell k}$ by (18).
8: **until** the objective value converges
9: **output:** Final beamforming matrix $\underline{\mathbf{V}}$

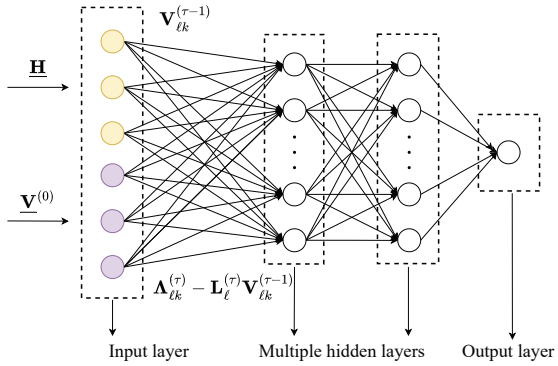


Fig. 1. The DNN structure used in the DeepFP network. The DNN consists of one input layer, multiple hidden layers, and one output layer. The activation function in the hidden layers is the complex extension of ReLU.

where

$$\widehat{\mathbf{V}}_{\ell k}=\mathbf{V}_{\ell k}^{(\tau-1)}+\frac{1}{\lambda_\ell}(\mathbf{\Lambda}_{\ell k}-\mathbf{L}_\ell\mathbf{V}_{\ell k}^{(\tau-1)}). \quad (19)$$

Here and throughout, we use the superscript τ or $\tau-1$ to index the iteration. The optimal updates of $\mathbf{Y}_{\ell k}$ and $\mathbf{\Gamma}_{\ell k}$ are the same as in (9) and (10). Algorithm 2 summarizes the FastFP method.

IV. Proposed DeepFP for Multicell MIMO Beamforming

Recall that the FastFP method requires computing the largest eigenvalue of $\mathbf{L}$ to decide each λ_ℓ, thus incurring a cubic computational complexity. This section introduces a deep unfolding method, called the DeepFP, that chooses λ_ℓ without explicit eigencomputation.

A. Deep Unfolding for Iterative Optimization

A generic iterative algorithm can be written in the following standard form as [6]

$$\underline{\mathbf{x}}^{(\tau)}=f_\tau(\underline{\mathbf{x}}^{(\tau-1)};\underline{\phi}), \quad (20)$$

where $\tau=1,2,\ldots,T$ denotes the iteration index, T is the total iteration number, $\underline{\mathbf{x}}$ is the optimization variable, the

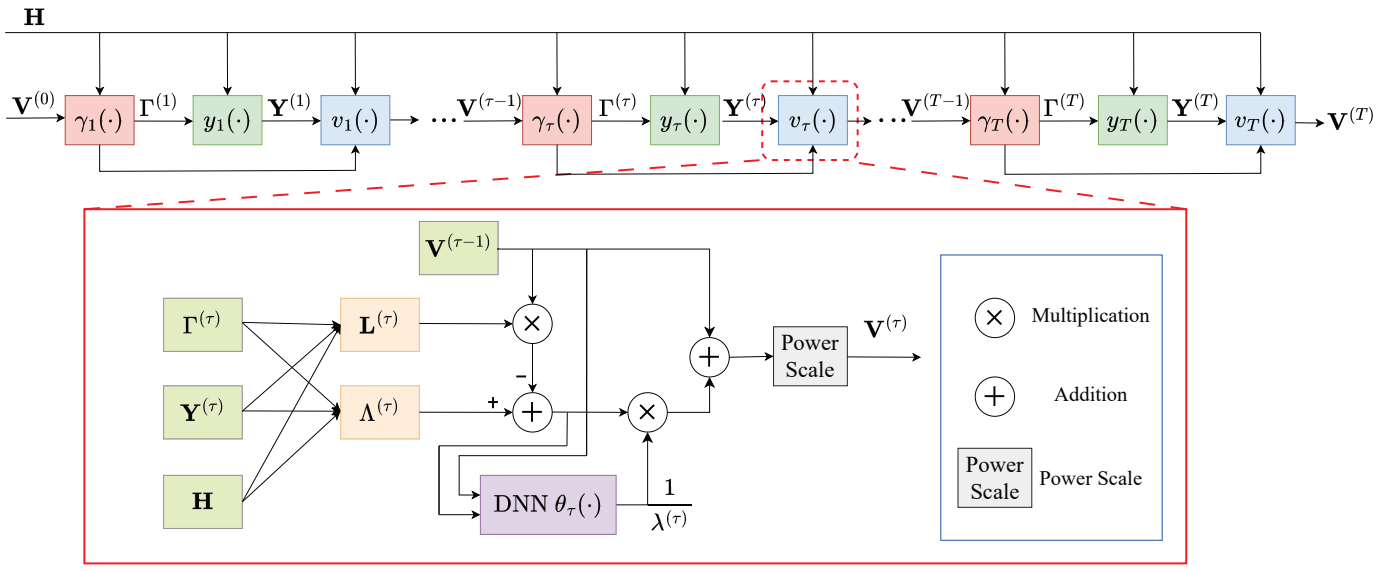


Fig. 2. The architecture of our proposed DeepFP network. The modules $z_\tau(\cdot), \gamma_\tau(\cdot), y_\tau(\cdot), v_\tau(\cdot)$ are designed based on (17), (10), (9), and (18), respectively. In the module $v_\tau(\cdot)$, the parameter $\lambda^{(\tau)}$ is provided by the DNN θ_τ, $\mathbf{L}^{(\tau)}$ is determined by (12), and $\mathbf{\Lambda}^{(\tau)}$ is determined by (8). The DNNs in different layers of the DeepFP network have the same structure but do not share parameters.

status variable $\underline{\phi}$ is a random variable that characterizes the uncertainty in the optimization problem (e.g., it is the random channel fading of the MIMO beamforming problem), and the f_τ is the iterate function that yields the new solution $\underline{\mathbf{x}}^{(\tau-1)}$ given the previous solution $\underline{\mathbf{x}}^{(\tau)}$ conditioned on the current status $\underline{\phi}$.

Deep Unfolding aims to unroll the iterative algorithm into a multi-layer sequential process. With a set of trainable parameters θ, the deep unfolding method represents (20) as a DNN layer:

$$\underline{\mathbf{x}}^{(\tau)} = \mathcal{F}_\tau(\underline{\mathbf{x}}^{(\tau-1)}; \theta_\tau, \underline{\phi}), \tag{21}$$

where $\tau = 1, 2, ..., T$ is reused to denote the layer index, $\mathcal{F}_\tau$ denotes the structure of deep unfolding network in the τth layer, and $\underline{\mathbf{x}}^{(\tau-1)}$ and $\underline{\mathbf{x}}^{(\tau)}$ are the input and output of the τth layer, respectively. In principle, after θ_τ has been trained properly, $\mathcal{F}_\tau(\underline{\mathbf{x}}^{(\tau-1)}; \theta_\tau, \underline{\phi})$ is expected to behave similarly to $f_\tau(\underline{\mathbf{x}}^{(\tau-1)}; \underline{\phi})$ for any possible $\underline{\phi}$.

B. Optimizing λ_ℓ via DNN

By specializing the above deep unfolding framework to the beamforming problem (4) and the FastFP algorithm, we have the following correspondence:

$$\underline{\mathbf{x}} = \{\mathbf{\Gamma}_{\ell k}, \mathbf{Y}_{\ell k}, \mathbf{V}_{\ell k}\}, \tag{22}$$

$$\underline{\phi} = \{\mathbf{H}_{\ell k, j}, w_{\ell k}, P_\ell, \sigma^2\}. \tag{23}$$

Equation (18) implies that the update of $\mathbf{V}_{\ell k}$ in FastFP follows a gradient projection form, where the scalar $1/\lambda_\ell$ serves as the step size for the update of all users in cell ℓ.

We treat $\lambda_{\ell k}^{(\tau)}$ as a function of $\mathbf{V}_{\ell k}^{(\tau-1)}$ and the term $\mathbf{\Lambda}_{\ell k}^{(\tau)} - \mathbf{L}_\ell^{(\tau)} \mathbf{V}_{\ell k}^{(\tau-1)}$. Let $\theta_\tau(\cdot)$ denote the τth DNN layer in the unfolding network. The value of $\lambda_{\ell k}^{(\tau)}$ is then given by

$$\lambda_{\ell k}^{(\tau)} = \theta_\tau(\mathbf{V}_{\ell k}^{(\tau-1)}, \mathbf{\Lambda}_{\ell k}^{(\tau)} - \mathbf{L}_\ell^{(\tau)} \mathbf{V}_{\ell k}^{(\tau-1)}). \tag{24}$$

As (24) indicates, we think of $\lambda_{\ell k}^{(\tau)}$ as a function of $\mathbf{V}_{\ell k}^{(\tau-1)}$ and $\mathbf{\Lambda}_{\ell k}^{(\tau)} - \mathbf{L}_\ell^{(\tau)} \mathbf{V}_{\ell k}^{(\tau-1)}$. Here is the rationale of the above setting: in the FastFP algorithm, the beamforming matrix $\mathbf{V}_{\ell k}^{(\tau)}$ is updated as a linear combination of its value from the previous iteration and a new direction matrix $\mathbf{\Lambda}_{\ell k}^{(\tau)} - \mathbf{L}_\ell^{(\tau)} \mathbf{V}_{\ell k}^{(\tau-1)}$. When the number of iterations is small, $\mathbf{V}_{\ell k}^{(\tau)}$ significantly deviates from the new direction. Thus, $\lambda_{\ell k}^{(\tau)}$ should be large to accelerate convergence. As the number of iterations increases, $\mathbf{V}_{\ell k}^{(\tau)}$ approaches the stationary point, and $\mathbf{\Lambda}_{\ell k}^{(\tau)} - \mathbf{L}_\ell^{(\tau)} \mathbf{V}_{\ell k}^{(\tau-1)}$ approaches the zero vector. In this case, $\lambda_{\ell k}^{(\tau)}$ should be small to avoid oscillations. Thus, it leads to modeling $\lambda_{\ell k}^{(\tau)}$ as a function of $\mathbf{V}_{\ell k}^{(\tau-1)}$ and $\mathbf{\Lambda}_{\ell k}^{(\tau)} - \mathbf{L}_\ell^{(\tau)} \mathbf{V}_{\ell k}^{(\tau-1)}$.

In the FastFP algorithm, $\lambda_{\ell k}^{(\tau)}$ is set to the largest eigenvalue of $\mathbf{L}_\ell$ to ensure convergence. In contrast, the proposed DeepFP network need not require $\lambda_{\ell k}^{(\tau)}$ to satisfy (14). Rather, our goal is seek a desirable $\lambda_{\ell k}^{(\tau)}$ through the DNN, to yield a better $\mathbf{V}_{\ell k}^{(\tau)}$. This goal can be achieved by choosing a smaller $\lambda_{\ell k}^{(\tau)}$ than that in (16). According to Majorization-minimization (MM) [33] theory, the WSR can be improved by optimizing its lower bound, i.e., the surrogate function $f_n(\underline{\mathbf{V}}, \underline{\mathbf{\Gamma}}, \underline{\mathbf{Y}}, \underline{\mathbf{Z}})$. A smaller $\lambda_{\ell k}^{(\tau)}$ may result in a tighter lower bound, thereby accelerating the iterative process.

The DNN structure used in the DeepFP network consists of one input layer, multiple hidden layers, and one output layer, as shown in Fig. 1. The input to the DNN is the flattened $\mathbf{V}_{\ell k}^{(\tau-1)}$ and $\mathbf{\Lambda}_{\ell k}^{(\tau)} - \mathbf{L}_\ell^{(\tau)} \mathbf{V}_{\ell k}^{(\tau-1)}$. Instead of dealing with the real and imaginary parts separately, we directly use flattened complex matrices as the integrated input to the DNN. To achieve this, we extend the Rectified Linear Unit (ReLU) [34] activation function to support complex-valued data in the hidden layers. Specifically, the complex ReLU is defined as:

$$\text{ReLU}_{\text{Complex}}(a + bi) = \max(a, 0) + \max(b, 0)i, \tag{25}$$

Algorithm 3 DeepFP for Multicell MIMO Beamforming

–Training Session–
input: Randomly generated channel samples.
Stage 1: Supervised Learning
Run Algorithm 2 to obtain the solution $\underline{\mathbf{V}}^*$.
Use $\underline{\mathbf{V}}^*$ as labels to train the DNN based on the loss function in (30).
Stage 2: Unsupervised Learning
Fine-tune the DNN parameters based on the loss function in (31).
output: The optimized DNN parameters.

–Test Session–
input: The current CSI.
initialize $\underline{\mathbf{V}}^{(0)}$ to feasible values under the power constraint.
for $\tau = 1$ to T **do**
  $\underline{\mathbf{\Gamma}}^{(\tau)} \leftarrow \gamma_\tau(\underline{\mathbf{V}}^{(\tau-1)}; \underline{\phi})$
  $\underline{\mathbf{Y}}^{(\tau)} \leftarrow y_\tau(\underline{\mathbf{V}}^{(\tau-1)}; \underline{\phi})$
  $\lambda_{\ell k}^{(\tau)} \leftarrow \theta_\tau(\mathbf{V}_{\ell k}^{(\tau-1)}, \mathbf{\Lambda}_{\ell k}^{(\tau)} - \mathbf{L}_\ell^{(\tau)} \mathbf{V}_{\ell k}^{(\tau-1)})$
  $\underline{\mathbf{V}}^{(\tau)} \leftarrow v_\tau(\underline{\mathbf{V}}^{(\tau-1)}, \underline{\mathbf{\Gamma}}^{(\tau)}, \underline{\mathbf{Y}}^{(\tau)}; \underline{\lambda}^{(\tau)}, \underline{\phi})$
end for
output: Beamforming matrices $\underline{\mathbf{V}}^{(T)}$.

where i is the imaginary unit. We use $\Re(\cdot)$ to denote the activation function in the output layer to ensure that the output of the DNN is a real number.

C. Unfolding Layers

With the DNN $\theta_\tau(\cdot)$, the structure of the τth layer in the DeepFP network can be described as

$$\underline{\mathbf{\Gamma}}^{(\tau)} = \gamma_\tau(\underline{\mathbf{V}}^{(\tau-1)}; \underline{\phi}), \tag{26}$$

$$\underline{\mathbf{Y}}^{(\tau)} = y_\tau(\underline{\mathbf{V}}^{(\tau-1)}; \underline{\phi}), \tag{27}$$

$$\lambda_{\ell k}^{(\tau)} = \theta_\tau(\mathbf{V}_{\ell k}^{(\tau-1)}, \mathbf{\Lambda}_{\ell k}^{(\tau)} - \mathbf{L}_\ell^{(\tau)} \mathbf{V}_{\ell k}^{(\tau-1)}), \tag{28}$$

$$\underline{\mathbf{V}}^{(\tau)} = v_\tau(\underline{\mathbf{V}}^{(\tau-1)}, \underline{\mathbf{\Gamma}}^{(\tau)}, \underline{\mathbf{Y}}^{(\tau)}; \underline{\lambda}^{(\tau)}, \underline{\phi}), \tag{29}$$

where (26), (27), and (29) correspond to the iterative algorithm steps (10), (9), and (18), respectively. Although the modules $\gamma_\tau(\cdot)$ and $y_\tau(\cdot)$ involve matrix inversion operations, but the matrices here are only $N_r \times N_r$, where N_r is the number of receive antennas at each user terminal. Since this paper focus on a massive MIMO network, N_r is typically much smaller than the number of transmit antennas N_t at each base station. For this reason, we do not give much attention to the $N_r \times N_r$ matrix inversion, but only focus on eliminating the $N_t \times N_t$ matrix inversion in the module $v_\tau(\cdot)$.

But what if N_r is also large? Actually, the elimination of the $N_r \times N_r$ matrix inversion is conceptually not different from that of the $N_t \times N_t$. As discussed in [14], we simply need to further apply the nonhomogeneous bound in Lemma 1 to the update of $\mathbf{Y}_{\ell k}$ and $\mathbf{\Gamma}_{\ell \mathbf{k}}$, although the math notation would be much more complicated.

The full structure of the DeepFP network is depicted in Fig. 2. The variables $\underline{\mathbf{L}}^{(\tau)}$ and $\underline{\mathbf{\Lambda}}^{(\tau)}$ are computed based on (12) and (8), respectively. The DNNs across different layers of the unfolding network are based on the same structure (e.g., the number of hidden layers, the number of neurons per layer, and the activation functions). The module named "Power Scale" represents scaling beamforming vectors to satisfy the power constraints. This block corresponds to (18). It aims to enforce the transmit power constraint for the beamforming matrix produced by the DNN.

Actually, the core question our paper aims to answer is whether it is worthwhile to stick to the MM properties. Indeed, the MM properties can warrant convergence, but at the cost of computation—we have to repeatedly compute the inverse of a large $N_t \times N_t$ matrix. The nonhomogeneous FP can eliminate the matrix inverse but then λ_ℓ is difficult to decide. It is shown in [14] that $\lambda_\ell = \lambda_{\max}(L_\ell)$ can guarantee the MM properties when the same λ_ℓ is used for all the users in cell ℓ, but then the performance hurts because it sacrifices the flexibility in choosing λ_ℓ. In contrast, this paper relaxes the MM constraint to allow λ_ℓ to be separately tuned for each individual user.

D. Training Strategy

We adopt a hybrid training strategy that comprises two stages. In the first stage, for a given channel sample $\underline{\mathbf{H}}$, let $\underline{\mathbf{V}}^*$ denote the solution obtained from the FastFP algorithm (Algorithm 2), and let $\underline{\mathbf{V}}^{(T)}$ denote the output of the unfolding network at the final layer. The first training stage employs supervised learning, with $\underline{\mathbf{V}}^*$ serving as the label with respect to the sample $\underline{\mathbf{H}}$. The MSE between $\underline{\mathbf{V}}^*$ and $\underline{\mathbf{V}}^{(T)}$ is adopted as the loss function in the first stage:

$$\mathrm{LOSS}_1 = \frac{1}{KL} \sum_{\ell=1}^{L} \sum_{k=1}^{K} \|\mathbf{V}_{\ell k}^{(T)} - \mathbf{V}_{\ell k}^*\|_2^2. \tag{30}$$

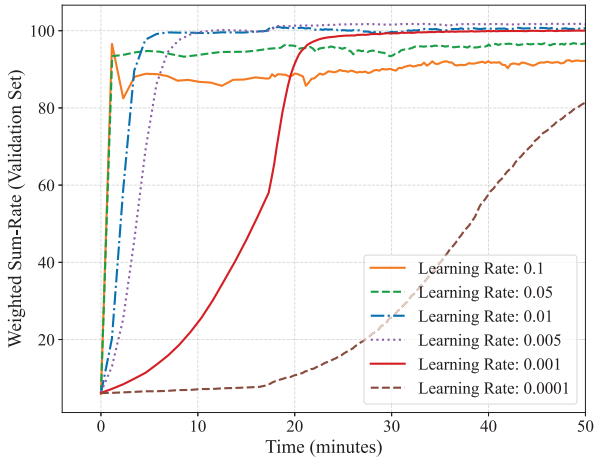
In the second stage, we switch to the unsupervised learning, using the WSR function of $\underline{\mathbf{V}}^{(T)}$ as the loss function, that is

$$\mathrm{LOSS}_2 = -\frac{1}{KL} \sum_{\ell=1}^{L} \sum_{k=1}^{K} w_{\ell k} R_{\ell k}. \tag{31}$$

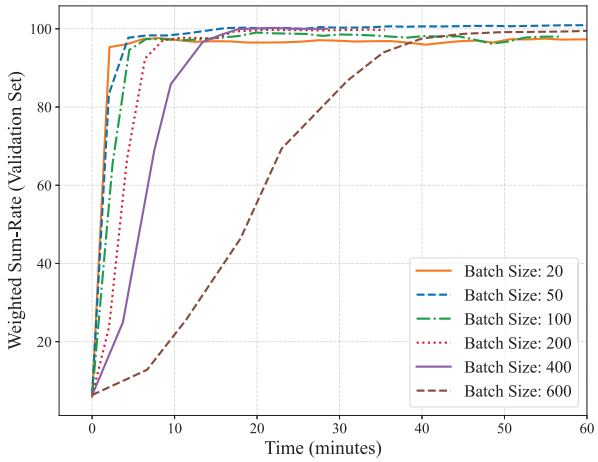
Although fully unsupervised learning has been shown to be feasible [35], we employs this hybrid training strategy that combines supervised and unsupervised learning.

Regarding the parameter initialization, each $\underline{\mathbf{V}}^{(0)}$ is randomly and independently generated according to the standard complex Gaussian distribution $\mathcal{CN}(0, 1)$, followed by a scaling process to meet the power constraint $\sum_{k=1}^{K} \mathrm{tr}(\mathbf{V}_{\ell k} \mathbf{V}_{\ell k}^H) = P_\ell$ for each BS. Moreover, for the supervised learning at the first stage, the FastFP algorithm and the unfolding network use the same starting point $\underline{\mathbf{V}}^{(0)}$. Algorithm 3 summarizes the training and inference procedures of the DeepFP method.

During the training stage, we use a large set of randomly generated CSI drawn from the same distribution to train the DNN. After training, the DNN is fixed and can be thought of as a deterministic function that takes the specific CSI values as input and yields the corresponding output. In other words, the DNN itself depends on the channel distribution rather than the specific CSI values. Thus, there is no need to retrain the DNN when the beamforming problem is considered for a new realization of CSI from the same distribution.



(a) Different choices of learning rate.



(b) Different choices of batch size.

Fig. 3. The WSR performance on validation dataset during training process for different learning rate (a) and batch size (b).

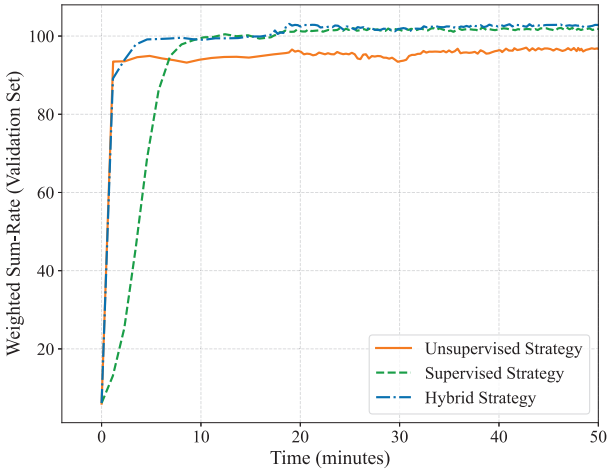


Fig. 4. Validation performance comparison of three training strategies: Unsupervised, Supervised, and Hybrid. The curves show the WSR over training time, with each strategy exhibiting distinct convergence behavior.

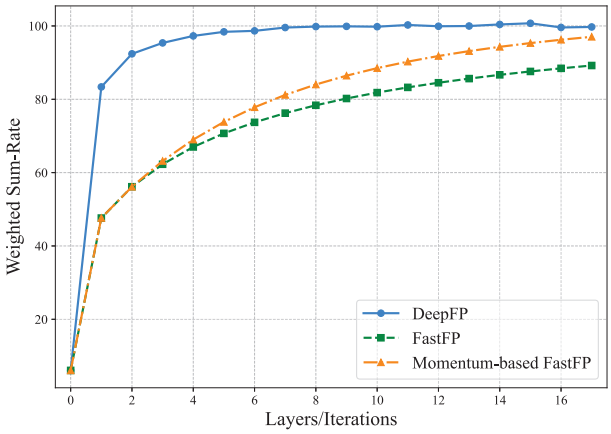


Fig. 5. Weighted sum-rate performance of the DeepFP network, the FastFP algorithm, and the momentum-based FastFP algorithm. For FastFP and momentum-based FastFP, the curves depict the WSR after i iterations. For DeepFP, the curve shows the WSR achieved by a network with i layers.

The current DeepFP requires centralized training, but there are several ways to make it distributed. One possible method is to group the cells into clusters and then train the DNN on a per-cluster basis. Another possible method is to incorporate federated learning into the training stage.

E. Complexity Analysis

Consider an L-cell MIMO system where each cell is equipped with N_t transmit antennas and serves K users. Each user is equipped with N_r receive antennas. The number of data streams is d. The computational complexity of Algorithm 2 at each iteration is given by

$$\mathcal{O}\left(LN_t^3 + L^2K^2(N_t+N_r)N_rd + LKN_r^2(N_r+d) + LK^2(N_t+d)d^2\right), \quad (32)$$

where the term $\mathcal{O}(LN_t^3)$ is for the computation of the largest eigenvalue in (16).

In the DeepFP network, the largest eigenvalue computation is replaced by the forward propagation of the DNN, with all other operations remaining unchanged. Consequently, the computational complexity for each layer is

$$\mathcal{O}\left(LKU(2N_td + (M_{\text{hid}}-1)U + 1) + L^2K^2(N_t+N_r)N_rd + LKN_r^2(N_r+d) + LK^2(N_t+d)d^2\right), \quad (33)$$

where M_{hid} represents the number of hidden layers in the DNN, and U represents the number of neural units in each hidden layer. Compared to the FastFP algorithm, the DeepFP network achieves lower computational complexity. It is worth emphasizing that the number of transmit antennas N_t at the base station is the dominant factor, especially considering a massive MIMO network. With respect to N_t, (32) shows that the eigenvalue-based benchmark method leads to a complexity of $O(N_t^3)$, while (33) shows that our method DeepFP leads to a much lower complexity of $O(N_t)$.

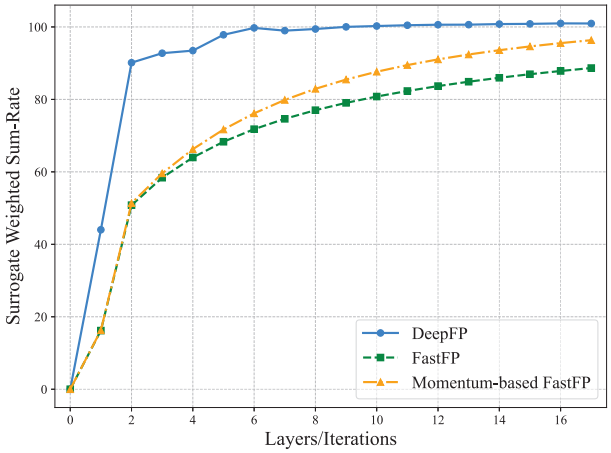


Fig. 6. Surrogate weighted sum-rate performance of the DeepFP network and the FastFP algorithm.

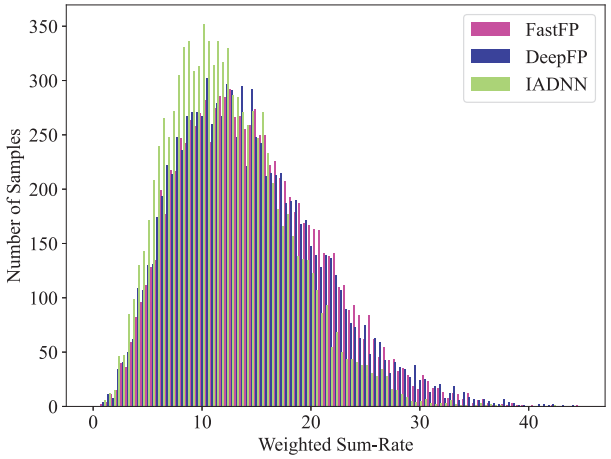


Fig. 7. Distributions of the DeepFP network and baseline algorithms in single cell MIMO system with $N_t = 64, N_r = 4, d = 2, K = 6$.

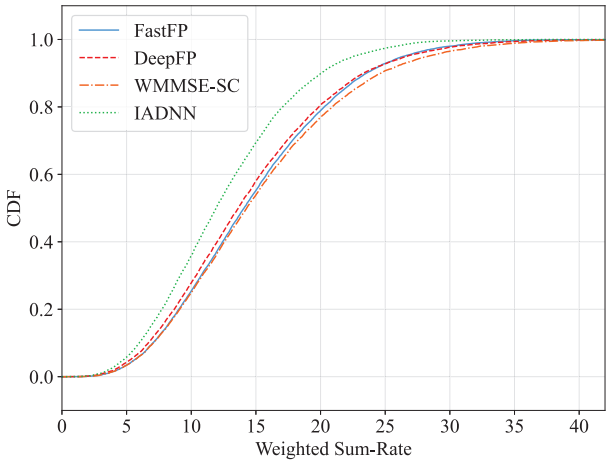


Fig. 8. The CDF that describes the rates achieved by different algorithms in single cell MIMO system with $N_t = 64, N_r = 4, d = 2, K = 6$.

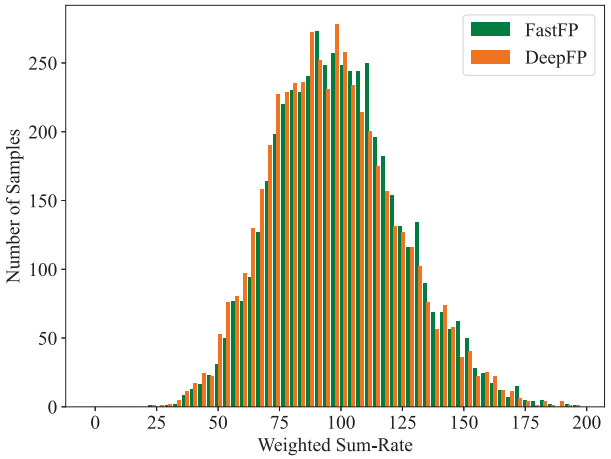


Fig. 9. Distributions of the DeepFP network and the FastFP algorithm in 7-cell MIMO with $N_t = 64, N_r = 4, d = 2, K = 6$.

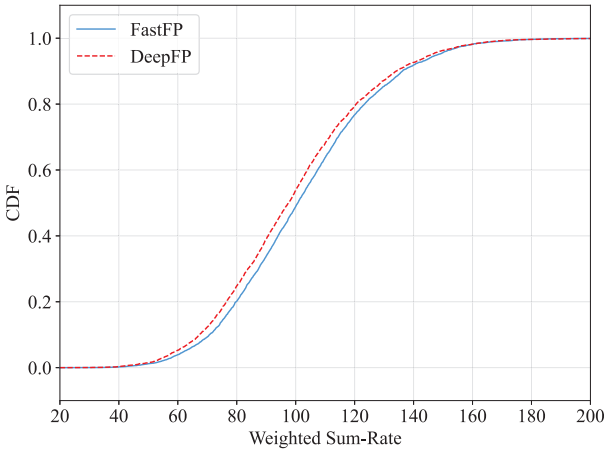


Fig. 10. The CDF that describes the rates achieved by different algorithms in 7-cell MIMO with $N_t = 64, N_r = 4, d = 2, K = 6$.

V. Experimental Results

In this section, we evaluate the performance of the proposed DeepFP network versus model-driven algorithms and existing unfolding algorithms. First, we evaluate how the different training strategies impact the optimization performance. Next, we try out a variety of wireless network examples. Finally, we validate the generalizability of the proposed DeepFP network by using different settings for training and test. The proposed DeepFP network is implemented in Python 3.10.0 with PyTorch 2.4.1. The system runs on a desktop with an Intel i7-13700 Central Processing Unit (CPU) clocked at 3.4 GHz and 64 GB of Random Access Memory (RAM). A Graphics Processing Unit (GPU) RTX 4080 is used during training to reduce training time, but not during testing. All algorithms, including DeepFP and the classical baselines, are implemented in PyTorch for consistency. For fair runtime comparison, all inference tests are conducted on CPU without GPU acceleration.

TABLE II
WEIGHTED SUM-RATE AND COMPUTATIONAL PERFORMANCE FOR SINGLE-CELL MIMO WITH $N_t = 64, N_r = 4, d = 2, K = 6$.

Algorithm	Weighted Sum-Rate	CPU Time (Sec.)
DeepFP	14.664 (98.9%)	0.053 (14.0%)
FastFP	14.826 (100.0%)	0.378 (100.0%)
FastFP (76 iterations)	14.664 (98.9%)	0.287 (76.0%)
WMMSE-SC	15.270 (103.0%)	0.563 (148.9%)
IADNN	12.540 (84.9%)	0.055 (14.5%)

TABLE III
WEIGHTED SUM-RATE AND COMPUTATIONAL PERFORMANCE FOR MULTICELL MIMO WITH $N_t = 64, N_r = 4, d = 2, K = 6$.

Algorithm	Weighted Sum-Rate	CPU Time (Sec.)
DeepFP	99.474 (97.4%)	0.275 (11.7%)
FastFP	102.186 (100.0%)	2.333 (100.0%)
FastFP (56 iterations)	99.480 (97.4%)	1.306 (56.0%)
GCN-WMMSE	91.011 (89.1%)	0.604(25.9%)

TABLE IV
WEIGHTED SUM-RATE AND COMPUTATIONAL PERFORMANCE OF THE DEEPFP NETWORK FOR MULTICELL MIMO WITH $N_t = 64, N_r = 4, d = 4$ FOR DIFFERENT K.

K	Weighted Sum-Rate	CPU Time (Sec.)	Iterations by FastFP
6	128.796 (92.8%)	0.285 (11.2%)	23
9	157.554 (90.3%)	0.584 (12.1%)	21
15	203.895 (86.5%)	1.678 (7.6%)	19

TABLE V
WEIGHTED SUM-RATE PERFORMANCE OF THE DEEPFP NETWORK WITH $N_t = 64, N_r = 4, K = 6$ FOR DIFFERENT d. THE DEEPFP NETWORK IS TRAINED WITH $d = 4$.

d	Weighted Sum-Rate (bit/sec.)	Iterations by FastFP
1	66.486 (96.5%)	57
2	97.074 (95.0%)	40
3	115.596 (94.6%)	32
4	128.796 (92.8%)	23

A. Experimental Setup

1) Dataset Generation: We generate channel data from a 7-hexagonal-cell MIMO system as considered in [14]. Within each cell, the BS is located at the center, and the K downlink users are randomly distributed. Each BS and user are equipped with N_t and N_r antennas, respectively. The number of data streams is $d \leq N_r$. The weights of all users are set to be equal. The distance between adjacent BSs is $D = 0.8$ km. The maximum transmit power of each BS is 20 dBm, and the background noise power is -90 dBm. The distance-dependent path loss of the downlink is modeled as $128.1+37.6\log_{10} r+\xi$ (in dB), where r denotes the distance from the BS to the user (in kilometers). ξ is a zero-mean Gaussian random variable with an 8 dB standard deviation to account for the shadowing effect.

2) Parameters Selection: For all our numerical results, the DNN consists of two hidden layers, one input layer, and one output layer. Unless explicitly stated, each hidden layer contains 64 neurons. We first investigate the impact of batch size and learning rate on convergence performance. We set $N_t = 64$, $N_r = 4$, $K = 6$, and $d = 2$. The DeepFP network has $T = 8$ layers. Fig.3 shows how the weighted sum-rate of the validation set changes during training for different batch size and learning rate settings. The results show that a larger learning rate speeds up convergence. However, an excessively large learning rate may cause instability and lower WSR performance. Thus, based on the results in Fig.3(a), we select an initial learning rate of 0.005, which is gradually decreased during the training process. The results in Fig. 3(b) show that as the batch size increases, the convergence rate initially improves and then decreases. This occurs because excessively large batch sizes result in longer processing times per minibatch due to memory limitations. Therefore, we choose a batch size of 200 to balance WSR performance and convergence rate. We further compare three training strategies: supervised, unsupervised, and hybrid. As shown in Fig. 4, the mixed training scheme can strike a better trade-off between the convergence speed and the ultimate performance than the benchmarks.

We then analyze the effect of the number of unfolding layers T in the DeepFP network on WSR performance. Networks with varying numbers of unfolding layers T are trained, and their WSR performance is evaluated on the test set. The results in Fig. 5 show that as T increases, the WSR performance improves initially but begins to fluctuate once T exceeds 8. Since the inference time of the DeepFP network grows linearly with T, we select $T = 8$ to balance WSR performance with inference time. Moreover, Fig. 5 demonstrates the significant performance advantage of the DeepFP network compared to the FastFP algorithm and the momentum-based FP. It shows that the DeepFP outperforms the benchmarks based on the MM properties. The DeepFP network achieves far superior performance compared to the FastFP algorithm when the number of layers in the DeepFP network equals the number of iterations in the FastFP algorithm. We further compare the surrogate function $f_n(\cdot)$ in (15) of DeepFP, FastFP and the momentum-based FastFP in Fig. 6. As shown, although the monotonic performance is no longer guaranteed by the DeepFP, the occasional performance drops are negligible overall, and the rate performance improves much more quickly thanks to the more aggressive choice of stepsizes.

B. WSR Maximization for Different Wireless Networks

1) Single-Cell Performance: We evaluate the WSR performance of the DeepFP network under different network sizes. We begin with a single-cell MU-MIMO system with $N_t = 64$, $N_r = 4$, $K = 6$, and $d = 2$. The following three algorithms are selected as baseline algorithms:

1) **FastFP Algorithm**: The result of the FastFP Algorithm is taken as the output of Algorithm 2 after 100 iterations.
2) **WMMSE-SC Algorithm**: The WMMSE-SC algorithm first uses WMMSE to solve a unconstrained WSR problem, and then scales the solution to satisfy the power constraints. This method avoids the bisection method but retains large matrix inversion, and it has theoretical guarantees only in the single-cell case. The result after

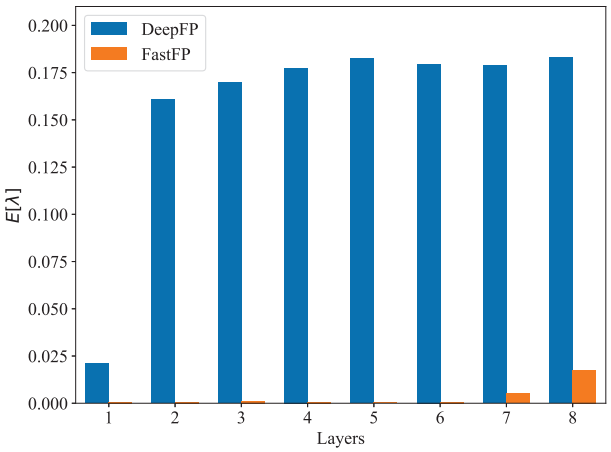


Fig. 11. The mean of λ selected by the FastFP algorithm and the DeepFP network. The FastFP algorithm computes λ based on (16), while λ in the the DeepFP network is determined by the DNNs.

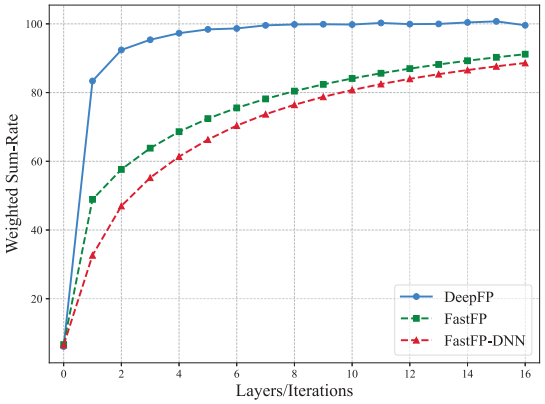


Fig. 12. Weighted sum-rate performance of the DeepFP network, the FastFP algorithm, and the FastFP-DNN algorithm.

TABLE VI
WEIGHTED SUM-RATE PERFORMANCE OF THE DEEPFP NETWORK WITH $N_r = 4, d = 2, K = 6$ FOR DIFFERENT N_t. THE DEEPFP NETWORK IS TRAINED WITH $N_t = 64$

N_t	Weighted Sum-Rate (bit/sec.)	Iterations by FastFP
16	55.224 (96.3%)	27
24	67.962 (96.7%)	39
32	77.502 (96.8%)	44
40	82.584 (96.5%)	42
48	86.880 (96.9%)	48
56	93.546 (96.8%)	49
64	99.474 (97.4%)	56

100 iterations is taken as the output of the WMMSE-SC algorithm.

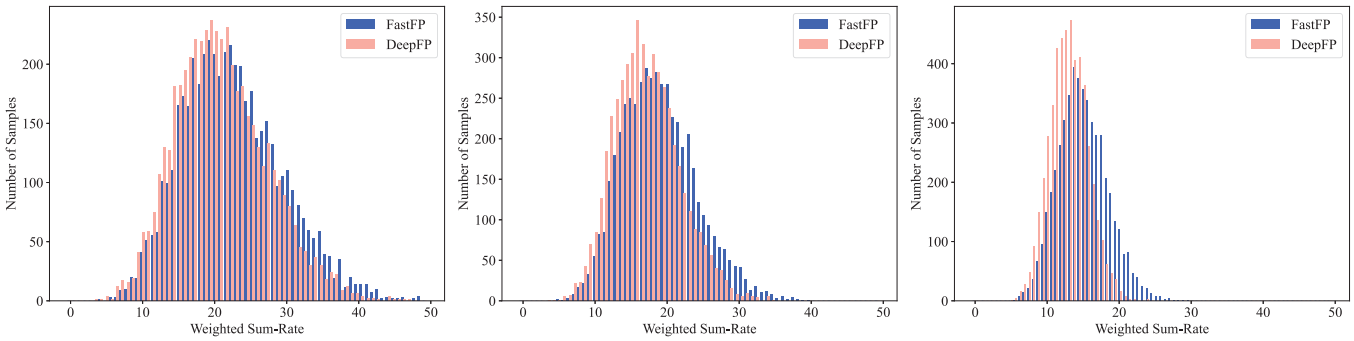
3) **IADNN**: The Iterative Algorithm-Induced Deep Unfolding Neural Network (IAIDNN) [6] unfolds the WMMSE-SC algorithm for single-cell MIMO systems. IAIDNN eliminates large matrix inversions by introducing trainable matrices that approximate matrix inversion based on the first-order Taylor expansion. We implemented the original network structure proposed in [6] using PyTorch, following the training settings recommended in [6]. The number of layers in IAIDNN is set to 7, as used in [6].

We evaluate the WSR performance of the DeepFP network and baseline algorithms using the same test data. The average WSR and CPU time are computed from $10,000$ test samples, with the results presented in Table II. We also report the results of FastFP after 76 iterations, which achieves the same WSR performance as the DeepFP network. The WSR performance and runtime of each algorithm are compared to those of the FastFP algorithm, using percentages for clarity. The results show that the DeepFP network achieves 98.9% of the WSR achieved by FastFP after 100 iterations, while using only 14.0% of its runtime. The FastFP algorithm requires 76 iterations to achieve the same WSR performance as the DeepFP network, resulting in nearly five times the runtime. Moreover, our algorithm outperforms IADNN in WSR performance with a similar computation time.

The distribution and cumulative distribution function (CDF) of the WSR performance achieved by different algorithms are shown in Fig.7 and Fig.8, respectively. Each distribution and its corresponding CDF are based on results from $10,000$ test samples. The results indicate that the proposed DeepFP network closely matches the performance of the FastFP algorithm and outperforms the IADNN algorithm.

2) Multicell Performance: We further validate the WSR performance of the DeepFP network in a 7-cell wrapped-around network. The settings are $N_t = 64$, $N_r = 4$, $K = 6$, and $d = 2$. The FastFP algorithm and the GCN-WMMSE algorithm [12] serves as two baselines. We reimplemented the GCN-WMMSE algorithm in PyTorch and adjusted the size of its graph neural network to match that of the DNN in DeepFP for a fair complexity comparison. Table III presents the WSR performance and CPU inference runtime. The results show that the proposed DeepFP network achieves 97.4% of the WSR attained by FastFP while requiring only 11.7% of its runtime. After 56 iterations, FastFP achieves the same performance as the DeepFP network. Since GCN-WMMSE is designed to mimic the performance of the traditional WMMSE algorithm, it inherently inherits the associated weaknesses, such as slow convergence. Compared to GCN-WMMSE, DeepFP not only achieves higher WSR performance but also incurs lower inference latency.Fig.9 and Fig.10 show the distribution and CDF of the WSR. The results indicate that the DeepFP network closely approximates the distribution of FastFP in the multicell MIMO system. Fig. 11 shows the mean of λ provided by the DNNs, as well as the mean of λ calculated using (16) under the same inputs. The results match our expectations: the DeepFP network produces smaller λ values.

To enhance the fairness of the experiments, we designed a new baseline algorithm, named FastFP-DNN. Specifically, we used a DNN to directly predict the maximum eigenvalue in FastFP, and designed the DNN size to ensure that it has identical computational complexity to DeepFP, so that the complexity is well normalized. The results shown in Fig. 12 demonstrate that the performance of the comparative algorithm is inferior to FastFP, while DeepFP achieves significantly superior performance compared to FastFP.



(a) $K = 6$ (b) $K = 9$ (c) $K = 15$

Fig. 13. Distributions of the DeepFP network and the FastFP algorithm in 7-cell MIMO with $N_t = 64, N_r = 4, d = 4$ for different K.

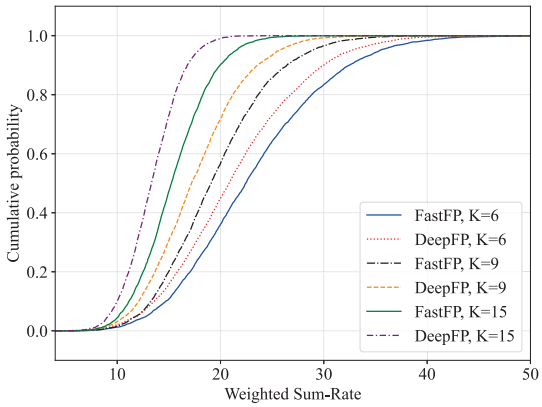


Fig. 14. The CDF that describes the rates achieved by the DeepFP network and the FastFP algorithm in 7-cell MIMO with $N_t = 64, N_r = 4, d = 4$ for different K.

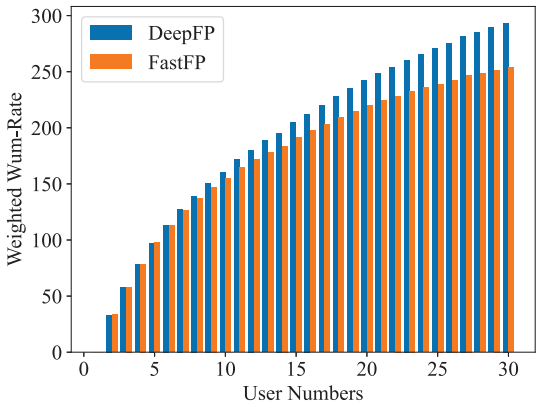


Fig. 15. The WSR performance of the DeepFP network and the FastFP algorithm in 7-cell MIMO with $N_t = 64, N_r = 4, d = 2$ for different K. The DeepFP network is trained with $N_t = 64, N_r = 4, d = 2, K = 6$.

Next, we consider scenarios with more users and higher data streams per user. We set $N_t = 64$, $N_r = 4$, $d = 4$, and $K = 6, 9, 15$. Table IV presents the average WSR performance and CPU time. We define "Iterations by FastFP" as the average number of iterations FastFP requires to achieve the same performance as the DeepFP network. The results show that as the number of users increases, the WSR performance improves. However, the gap between the DeepFP network and FastFP also widens. Comparing Table IV with Table III, when $N_t = 64$, $N_r = 4$, and $K = 6$, the DeepFP network demonstrates better acceleration performance at $d = 2$. Fig.13 and Fig.14 show the distribution and CDF of the WSR for different values of K, respectively. The results indicate that although the WSR performance of the DeepFP network decreases in percentage terms with an increasing number of users, it still provides a good approximation of the distribution of the FastFP algorithm.

C. Generalizability Validation

In the previous subsection, we evaluated the WSR and acceleration performance of the DeepFP network in MIMO systems of varying sizes. In practice, the test data often differs significantly from the training data, which arises from two aspects: 1) The test data may differ in size from the training data. For instance, in multicell MIMO, the number of users may change due to mobility. Additionally, we expect the trained network to be applicable to data from different cells, leading to variations in the number of transmit antennas. 2) Changes in the data distribution. Even if the test data has the same size as the training data, its distribution may differ. Therefore, in this subsection, we assess the generalization performance of the DeepFP network.

First, we use the network trained with $N_t = 64$, $N_r = 4$, $K = 6$, and $d = 4$ to test its performance under different values of d. The results are shown in Table V. These results indicate that the trained DeepFP network still performs well in terms of WSR for different values of d. As d decreases, the number of iterations required by FastFP to achieve the same performance increases. Comparing the results for $d = 2$ with those in Table III, the network's WSR performance decreases by 2.4% when the number of data streams per user increases.

Next, we test the performance of the DeepFP network, trained with $N_t = 64$, $N_r = 4$, $K = 6$, and $d = 2$, under different values of N_t. The results are shown in Table VI. These results indicate that as N_t changes, the WSR performance of the DeepFP network remains stable at around 97%.

Next, we continue using the DeepFP network trained with

TABLE VII
WEIGHTED SUM-RATE PERFORMANCE OF THE DEEPFP NETWORK FOR DIFFERENT CELL DISTANCES D AND STANDARD DEVIATION OF ξ: THE NETWORK IS TRAINED FOR $D = 0.8$ AND A STANDARD DEVIATION OF $\xi = 8$.

Cell Distance (km)	Weighted Sum-Rate (bit/sec.)			Iterations by FastFP		
	ξ 4 dB	ξ 8 dB	ξ 12 dB	ξ 4dB	ξ 8 dB	ξ 12 dB
0.4	261.168 (93.2%)	261.012 (101.5%)	259.386 (113.2%)	58	132	233
0.6	167.178 (94.8%)	186.222 (99.1%)	205.962 (109.5%)	45	91	197
0.8	108.090 (96.1%)	136.746 (99.2%)	162.756 (107.5%)	35	89	152
1.0	73.386 (96.5%)	100.506 (99.1%)	127.536 (105.2%)	25	82	217
1.2	52.062 (97.0%)	76.446 (98.8%)	105.948 (103.5%)	21	71	204

TABLE VIII
SUM-RATE PERFORMANCE OF DEEPFP AND FASTFP IN A RAYLEIGH FADING CHANNEL: THE NETWORK IS TRAINED UNDER A SHADOWING MODEL AND TESTED ON RAYLEIGH FADING WITHOUT SHADOWING.

Transmit Power (dB)	Weighted Sum-Rate (bit/sec.)	
	FastFP	DeepFP
0	56.7086	68.4735
10	62.5130	70.0218
20	66.9725	70.4233
30	71.3865	71.4206
40	76.2021	73.0454

$N_t = 64$, $N_r = 4$, $K = 6$, and $d = 2$ to test its WSR performance on datasets with varying numbers of users. The results are shown in Fig. 15. These results indicate that when $K < 6$, the DeepFP network outperforms the FastFP algorithm after 100 iterations. As K increases, the gap between the DeepFP network and the FastFP algorithm widens.

In the previous generalization test, we evaluated the trained network's generalization ability on test data of different sizes. Next, we test the network's generalization performance on data with different distributions. During the generation of the training data, the distance between base stations is set to $D = 0.8$ km, and the standard deviation of the path loss parameter ξ is 8 dB. We generate test data with varying values of D and standard deviation, and then evaluate the network's WSR performance. The results are presented in Table VII. These results show that the DeepFP network demonstrates good WSR performance under different distributions. To further assess generalization under distinct channel statistics, we evaluate the network in a Rayleigh fading scenario, trained under shadowing conditions. As shown in Table VIII, the DNN trained for random shadowing can still yield comparable performance in the Rayleigh fading model.

VI. CONCLUSION

This work aims at a novel deep unfolding paradigm for optimizing the multicell MIMO beamformers in cellular networks. The proposed DeepFP method can be distinguished from the existing deep unfolding methods [6]–[11] for MIMO beamforming in two respects. First, while the previous work [22] can only reduce the complexity of large matrix inversion, DeepFP eliminates the large matrix inversion completely. Second, while the previous work can linearize the Lagrange multiplier optimization only for a single cell, DeepFP extends the linearization for a generic multicell network. DeepFP acquires the above two benefits by linking the traditional WMMSE algorithm [2], [3] with the FP tools [4], [5] and further incorporating an inhomogeneous bound [14] into the DNN design for deep unfolding. Extensive numerical examples show that DeepFP reduces the complexity of the conventional model-driven iterative algorithms (such as WMMSE) and can even outperform them in maximizing the WSR for multicell MIMO networks.

REFERENCES

[1] J. Zhu, T.-H. Chang, L. Xiang, and K. Shen, "DeepFP: Deep-unfolded fractional programming for massive MIMO beamforming," in *Proc. IEEE Int. Workshop Signal Process. Adv. Wireless Commun. (SPAWC)*, June 2025.

[2] Q. Shi, M. Razaviyayn, Z.-Q. Luo, and C. He, "An iteratively weighted MMSE approach to distributed sum-utility maximization for a MIMO interfering broadcast channel," *IEEE Trans. Signal Process.*, vol. 59, no. 9, pp. 4331–4340, Sept. 2011.

[3] S. S. Christensen, R. Agarwal, E. De Carvalho, and J. M. Cioffi, "Weighted sum-rate maximization using weighted MMSE for MIMO-BC beamforming design," *IEEE Trans. Wireless Commun.*, vol. 7, no. 12, pp. 4792–4799, Dec. 2008.

[4] K. Shen and W. Yu, "Fractional programming for communication systems—Part I: Power control and beamforming," *IEEE Trans. Signal Process.*, vol. 66, no. 10, pp. 2616–2630, May 2018.

[5] ——, "Fractional programming for communication systems—Part II: Uplink scheduling via matching," *IEEE Trans. Signal Process.*, vol. 66, no. 10, pp. 2631–2644, May 2018.

[6] Q. Hu, Y. Cai, Q. Shi, K. Xu, G. Yu, and Z. Ding, "Iterative algorithm induced deep-unfolding neural networks: Precoding design for multiuser MIMO systems," *IEEE Trans. Wireless Commun.*, vol. 20, no. 2, pp. 1394–1410, Feb. 2021.

[7] M. Zhu, T.-H. Chang, and M. Hong, "Learning to beamform in heterogeneous massive MIMO networks," *IEEE Trans. Wireless Commun.*, vol. 22, no. 7, pp. 4901–4915, July 2023.

[8] L. Pellaco, M. Bengtsson, and J. Jaldén, "Matrix-inverse-free deep unfolding of the weighted MMSE beamforming algorithm," *IEEE Open J. Commun. Soc.*, vol. 3, pp. 65–81, Dec. 2022.

[9] N. T. Nguyen, M. Ma, O. Lavi, N. Shlezinger, Y. C. Eldar, A. L. Swindlehurst, and M. Juntti, "Deep unfolding hybrid beamforming designs for THz massive MIMO systems," *IEEE Trans. Signal Process.*, vol. 71, pp. 3788–3804, Oct. 2023.

[10] Q. Hu, Y. Liu, Y. Cai, G. Yu, and Z. Ding, "Joint deep reinforcement learning and unfolding: Beam selection and precoding for mmWave multiuser MIMO with lens arrays," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 8, pp. 2289–2304, Aug. 2021.

[11] C. Xu, Y. Jia, S. He, Y. Huang, and D. Niyato, "Joint user scheduling, base station clustering, and beamforming design based on deep unfolding technique," *IEEE Trans. Commun.*, vol. 71, no. 10, pp. 5831–5845, Oct. 2023.

[12] L. Schynol and M. Pesavento, "Coordinated sum-rate maximization in multicell MU-MIMO with deep unrolling," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 4, pp. 1120–1134, Apr. 2023.

[13] A. Chowdhury, G. Verma, A. Swami, and S. Segarra, "Deep graph unfolding for beamforming in MU-MIMO interference networks," *IEEE Trans. Wireless Commun.*, vol. 23, no. 5, pp. 4889–4903, May 2024.

[14] K. Shen, Z. Zhao, Y. Chen, Z. Zhang, and H. V. Cheng, "Accelerating quadratic transform and WMMSE," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 11, p. 3110–3124, July 2024.

[15] Z.-Q. Luo and S. Zhang, "Dynamic spectrum management: Complexity and duality," *IEEE J. Sel. Top. Signal Process.*, vol. 2, no. 1, pp. 57–73, Feb. 2008.

[16] S. Joshi, P. C. Weeraddana, M. Codreanu, and M. Latva-aho, "Weighted sum-rate maximization for MISO downlink cellular networks via branch and bound," in *2011 Conf. Rec. Forty Fifth Asilomar Conf. Signals, Syst. Comput. (ASILOMAR)*, Apr. 2011, pp. 1569–1573.

[17] L. Liu, R. Zhang, and K.-C. Chua, "Achieving global optimality for weighted sum-rate maximization in the K-user Gaussian interference channel with multiple antennas," *IEEE Trans. Wireless Commun.*, vol. 11, no. 5, pp. 1933–1945, May 2012.

[18] A. Kammoun, A. Müller, E. Björnson, and M. Debbah, "Linear precoding based on polynomial expansion: Large-scale multi-cell MIMO systems," *IEEE J. Sel. Top. Signal Process.*, vol. 8, no. 5, pp. 861–875, Oct. 2014.

[19] X. Gao, O. Edfors, F. Rusek, and F. Tufvesson, "Linear pre-coding performance in measured very-large MIMO channels," in *2011 IEEE Veh. Technol. Conf. (VTC Fall)*, Dec. 2011, pp. 1–5.

[20] L. D. Nguyen, H. D. Tuan, T. Q. Duong, and H. V. Poor, "Multi-user regularized zero-forcing beamforming," *IEEE Trans. Signal Process.*, vol. 67, no. 11, pp. 2839–2853, June 2019.

[21] D. P. Bertsekas, *Nonlinear Programming*. Cambridge, MA, USA: MIT Press, 1999.

[22] X. Zhao, S. Lu, Q. Shi, and Z.-Q. Luo, "Rethinking WMMSE: Can its complexity scale linearly with the number of BS antennas?" *IEEE Trans. Signal Process.*, vol. 71, pp. 433–446, Feb. 2023.

[23] Z. Zhang, Z. Zhao, and K. Shen, "Enhancing the efficiency of WMMSE and FP for beamforming by minorization-maximization," in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process. (ICASSP)*, May 2023.

[24] R. Sun, C. Wang, A.-A. Lu, X. Fu, X. Liu, Y. Zhang, X. Gao, and X.-G. Xia, "Matrix manifold precoder design for massive MIMO downlink," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, July 2024, pp. 1–6.

[25] H. Sun, X. Chen, Q. Shi, M. Hong, X. Fu, and N. D. Sidiropoulos, "Learning to optimize: Training deep neural networks for interference management," *IEEE Trans. Signal Process.*, vol. 66, no. 20, pp. 5438–5453, Oct. 2018.

[26] W. Cui, K. Shen, and W. Yu, "Spatial deep learning for wireless scheduling," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 6, pp. 1248–1261, June 2019.

[27] W. Xia, G. Zheng, Y. Zhu, J. Zhang, J. Wang, and A. P. Petropulu, "A deep learning framework for optimization of MISO downlink beamforming," *IEEE Trans. Commun.*, vol. 68, no. 3, pp. 1866–1880, Mar. 2020.

[28] H. He, C.-K. Wen, S. Jin, and G. Y. Li, "Deep learning-based channel estimation for beamspace mmWave massive MIMO systems," *IEEE Wireless Commun. Lett.*, vol. 7, no. 5, pp. 852–855, Oct 2018.

[29] K. Gregor and Y. LeCun, "Learning fast approximations of sparse coding," in *Int. Conf. Mach. Learn. (ICML)*, June 2010, pp. 399–406.

[30] J. R. Hershey, J. L. Roux, and F. Weninger, "Deep unfolding: Model-based inspiration of novel deep architectures," 2014. [Online]. Available: https://arxiv.org/abs/1409.2574

[31] A. Balatsoukas-Stimming and C. Studer, "Deep unfolding for communications systems: A survey and some new directions," in *2019 IEEE Workshop Signal Process. Syst. (SiPS)*, Mar. 2019, pp. 266–271.

[32] A. Goldsmith, *Wireless Communications*. Cambridge, U.K.: Cambridge University Press, 2005.

[33] Y. Sun, P. Babu, and D. P. Palomar, "Majorization-minimization algorithms in signal processing, communications, and machine learning," *IEEE Trans. Signal Process.*, vol. 65, no. 3, pp. 794–816, Feb. 2017.

[34] A. F. Agarap, "Deep learning using rectified linear units (ReLU)," 2019. [Online]. Available: https://arxiv.org/abs/1803.08375

[35] H. Hojatian, J. Nadal, J.-F. Frigon, and F. Leduc-Primeau, "Unsupervised deep learning for massive MIMO hybrid beamforming," *IEEE Trans. Wireless Commun.*, vol. 20, no. 11, pp. 7086–7099, May 2021.

Jianhang Zhu (Student Member, IEEE) received the B.E. degree in communication engineering from the School of Electronics and Information Technology, Sun Yat-sen University, Guangzhou, China, in 2021, and the M.E. degree in computer science from the School of Computer Science and Engineering, Sun Yat-sen University, in 2024. He is currently pursuing the Ph.D. degree with the School of Science and Engineering, The Chinese University of Hong Kong (Shenzhen), China. His research interests include semantic communication, deep unfolding, non-convex optimization, machine learning, the Age of Information, edge computing, and the Internet of Things.

Tsung-Hui Chang (Fellow, IEEE) received the B.S. degree in electrical engineering and the Ph.D. degree in communications engineering from the National Tsing Hua University (NTHU), Hsinchu, Taiwan, in 2003 and 2008, respectively. Currently, he is a Professor and the Associate Dean (Education) of the School of Artificial Intelligence, The Chinese University of Hong Kong, Shenzhen (CUHK-SZ), China, and the Associate Director of Guangdong Provincial Key Laboratory of Big Data Computing. Before joining CUHK-SZ, he was with the National Taiwan University of Science and Technology (NTUST), the University of California, Davis, as a Postdoctoral Researcher and a Faculty Member, respectively. His research interests include signal processing and optimization problems in data communications and machine learning. He has been an Elected Member of the IEEE Signal Processing Society (SPS) Signal Processing for Communications and Networking Technical Committee (SPCOM TC) since 2020. He received the Young Scholar Research Award of NTUST in 2014, the IEEE ComSoc Asian-Pacific Outstanding Young Researcher Award in 2015, the IEEE SPS Best Paper Awards in 2018 and 2021, the Outstanding Faculty Research Award of SSE of CUHK-SZ in 2021, and the Outstanding Research Award of CUHK-SZ in 2024. He is the Founding Chair of the IEEE SPS Integrated Sensing and Communication Technical Working Group (ISAC TWG) and the elected Regional Director-atLarge of Board of Governors of IEEE SPS from 2022 to 2023. He has served on the editorial board for major SP journals, including an Associate Editor for IEEE TRANSACTIONS ON SIGNAL PROCESSING from 2014 to 2018, IEEE TRANSACTIONS ON SIGNAL AND INFORMATION PROCESSING OVER NETWORKS from 2015 to 2018, IEEE OPEN JOURNAL OF SIGNAL PROCESSING since 2020, and a Senior Area Editor for IEEE TRANSACTIONS ON SIGNAL PROCESSING from 2021 to 2025.

Liyao Xiang (Member, IEEE) received the B.Eng. degree in Electrical and Computer Engineering from Shanghai Jiao Tong University, Shanghai, China, in 2012, and the Ph.D. degree in Computer Engineering from the University of Toronto, Toronto, ON, Canada, in 2018. She is currently an associate professor at Shanghai Jiao Tong University. Her research interests include AI security and privacy, privacy analysis in data mining, and mobile computing.

Kaiming Shen (Senior Member, IEEE) received the B.Eng. degree in information security and the B.Sc. degree in mathematics from Shanghai Jiao Tong University, China in 2011, and then the M.A.Sc. degree in electrical and computer engineering from the University of Toronto, Canada in 2013. After working at a tech startup in Ottawa for one year, he returned to the University of Toronto and received the Ph.D. degree in electrical and computer engineering in early 2020. He has been with the School of Science and Engineering at The Chinese University of Hong Kong, Shenzhen, China as a tenure-track assistant professor since 2020. His research interests include optimization, wireless communications, and information theory. He currently serves as an Editor for IEEE Transactions on Wireless Communications. He is a member of the Signal Processing for Communications and Networking Technical Committee of the IEEE Signal Processing Society. He received the IEEE Signal Processing Society Young Author Best Paper Award in 2021, the University Teaching Achievement Award in 2023, the Frontiers of Science Award in 2024, and the Chinese Information Theory Society Young Researcher Award in 2025.